

ARRAY STORY CHALLENGES

Practice problems on one-dimensional and two-dimensional arrays

1. The Collector's Storage Audit

A collector has several storage boxes arranged in a row. Each box contains one item, represented by a number from **0 to 9**. The collector wants to know which items have multiple copies, how many copies exist, and exactly where each copy is stored.

Given the contents of n boxes, write a C program that:

- identifies every item value that occurs more than once;
- prints the number of copies of each repeated item;
- prints all box positions containing that item; and
- prints the number of distinct item values that are repeated.

Sample input

```
Number of boxes: 10
Items: 7 3 7 2 3 7 5 2 9 5
```

Possible output

```
Item 2: 2 copies; boxes 4, 8
Item 3: 2 copies; boxes 2, 5
Item 5: 2 copies; boxes 7, 10
Item 7: 3 copies; boxes 1, 3, 6
Distinct repeated item values: 4
```

***Efficiency requirement:** The overall running time must be $O(n)$. Since item values are restricted to 0-9, auxiliary arrays indexed by the item value may be used.*

Cases worth testing: no repeated items, every box containing the same item, and some values from 0 to 9 not appearing at all.

2. The Treasure Map Survey

An explorer has converted an ancient treasure map into a rectangular grid. Each cell contains one of three values:

- **0** - empty land
- **1** - treasure
- **-1** - trap

Given an $m \times n$ map, write a C program that:

- counts the total number of treasures and traps;
- finds the row containing the greatest number of treasures;
- prints the position of every treasure; and
- checks whether any treasure is surrounded by traps in all four directions: up, down, left, and right.

Sample map

0	1	0	0
-1	1	-1	0
0	-1	0	1
0	0	-1	0

A treasure on the boundary cannot be surrounded on all four sides because one or more required neighbours do not exist. If multiple rows tie for the maximum number of treasures, print all of them.

Optional extension: For every treasure, count the number of adjacent traps using only the four-direction rule.

3. The Train Seat Puzzle

A train compartment has seats arranged in a single row. A value of **1** means that a seat is occupied, while **0** means that it is empty. A new passenger wants an empty seat that is as far as possible from the nearest occupied seat.

Given the seat arrangement, write a C program that:

- finds the empty seat whose distance from the nearest occupied seat is maximum;
- prints that maximum distance; and
- prints all equally good seats when more than one seat gives the same maximum distance.

Sample input

```
Seats: 1 0 0 0 1 0 1
```

Possible output

```
Best seat: 3
Distance from nearest occupied seat: 2
```

Seats at the two ends require special attention because an empty end seat may have occupied seats on only one side.

Cases worth testing: the best seat is at an end, several seats tie, no empty seat exists, and no occupied seat exists.